Studio Nx

Backend(FastAPI/Python) 78 — is_api_request()

2026-10-12

Backend(FastAPI/Python) 78 — `is_api_request()` (API 요청인지 가르기)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: 오류가 났을 때 JSON으로 답할지, 오류 페이지 HTML로 답할지 정하는 첫 단계 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	(P1-22 첫 함수) · 79 `error_page()` (다음)

1. 하는 일

요청의 주소를 보고 **API 요청인지, 화면(페이지) 요청인지** 알려 줍니다. 오류 처리기가 이 결과로 응답 형식을 고릅니다.

요청한 곳	결과	오류가 나면
`fetch('/api/v1/me/dashboard')` 같은 JS의 API 호출	`True`	지금처럼 JSON (`{"error": {...}}`)
브라우저 주소창에 입력한 `/myPage.html`, `/no-such-page`	`False`	오류 페이지 HTML (예: `errors/500.html`)

JS는 JSON의 `error.code`로 분기하므로 API에 HTML을 돌려주면 안 되고, 사람이 주소창으로 연 화면에는 JSON 글자 대신 오류 페이지를 보여 줘야 하기 때문에 둘을 가릅니다.

2. 언제 쓰나

호출하는 곳	용도
80 `http_error_handler` (다음 이후)	404, 405 등 기본 오류에서 HTML과 JSON 중 고르기
81 `unexpected_error_handler`	예상 못 한 500에서 HTML과 JSON 중 고르기

3. 코드

import (이번에 추가한 것만)

```
from app.core.config import get_settings
```

함수

```
def is_api_request(request: Request) -> bool:
    prefix = get_settings().api_prefix
    path = request.url.path
    return path == prefix or path.startswith(prefix + "/")
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻	예 (`prefix = "/api/v1"`)
`get_settings().api_prefix`	설정에 적힌 API 주소 앞부분입니다. 코드에 `"/api/v1"`을 직접 쓰지 않고 설정을 따릅니다	`"/api/v1"`
`request.url.path`	요청 주소에서 쿼리(`?a=1`)를 뺀 경로입니다	`/home.html?next=/api/v1/me` → `/home.html`
`path == prefix`	API 맨 앞 주소 자체입니다	`/api/v1`
`path.startswith(prefix + "/")`	API 아래의 주소입니다	`/api/v1/me`

4. 흐름

1. 설정에서 API 앞부분을 읽습니다.
2. 요청 경로가 그 앞부분과 같거나, 앞부분 뒤에 `/`가 붙어서 이어지면 `True`입니다.
3. 그 밖의 모든 경로는 `False`입니다.

5. 왜 이렇게

선택	이유
`startswith(prefix)`만 쓰지 않고 `prefix + "/"`로 경계를 확인	`/api/v10`, `/api/v1x` 같은 주소를 API로 잘못 보지 않기 위해서입니다
쿼리 문자열은 보지 않음	`request.url.path`에는 쿼리가 없어서, `/home.html?next=/api/v1/me`가 API로 오해되지 않습니다
설정값(`api_prefix`)을 읽음	`main.py`가 같은 값으로 라우터를 붙이므로 한 곳만 바꾸면 됩니다
헤더(`Accept`)로 가르지 않음	주소로 가르는 쪽이 단순하고, `SiteFiles`(없는 주소 404)도 같은 기준이라 서로 맞습니다
`core/site_files.py`는 그대로	그곳은 `startswith(prefix)`만 쓰는 간단한 판단이며, 이번 작업 범위 밖입니다

6. 시험 (`tests/test_is_api_request.py`, 18개)

시험	결과
API 주소 5개 (`/api/v1`, `/api/v1/`, `/api/v1/me`, `/api/v1/projects/12`, 없는 API)	모두 `True`
화면 주소 7개 (`/`, `/home.html`, `/myPage.html`, `/errors/500.html`, 없는 페이지, `/admin`, `/docs`)	모두 `False`
비슷한 주소 5개 (`/api/v10`, `/api/v1x/me`, `/api`, `/api/v2/me`, `/x/api/v1/me`)	모두 `False`
쿼리에 `/api/v1`이 있는 화면 주소, 쿼리가 붙은 API 주소	`False`, `True`

18개 모두 통과했습니다. 백엔드 전체 시험은 마지막 함수까지 끝낸 뒤 한 번에 돌립니다. `ruff`는 통과했습니다.